Requirements Analysis and Design (RAD), Sem01_2627

Project Proposal for SaigonBite

Online Food Delivery Hub Platform (OFDH)

Part 1 of the RAD Final Project

Team member	Student ID	Role	Owned main use cases
Nguyen Thanh Dat (Dat)	722I0004	Ordering, hub management and integration lead. Keeps the vision, backlog, risk list, Assumption Log and the business process diagram (Figure 1). Owns the Order and Hub state machines.	UC-01 Place an order; UC-03 Manage hubs and kitchen capacity
Le Minh Quan (Quan)	523K0049	Fulfilment and reporting. Keeps the schema, seed data and domain model. Owns the Delivery state machine.	UC-02 Deliver an order; UC-04 Produce management reports

Team: Team02

Date: 25 October 2026

File: Team02_Proposal_for_SaigonBite.pdf

Contents

1. System vision 2
1.1 The problem 2
1.2 Vision 3
1.3 Capabilities and expected benefits 3
1.4 Constraints fixed by the case 3
2. Approach and technology 4
2.1 Iterative Unified Process 4
2.2 Technology stack (decided) 4
2.3 Three-layer design and the four roles 4
3. Overall business process 5
3.1 Figure 1 in five parts 6
4. Stakeholders, actors and use case ownership 11
4.1 Stakeholders and actors 11
4.2 Use case ownership 11
5. Scope 12
5.1 CRUD note 12
5.2 Integration demo scenario 13
6. Six-iteration roadmap 13
6.1 Gantt chart 14
6.2 Planned effort and buffer 15
7. Iteration 1 in detail 15
7.1 Day-by-day plan 15
7.2 Iteration 1 acceptance criteria (planned) 16
7.3 Plan vs actual 16
8. Prioritised backlog 17
9. Top 5 risks 18
10. Assumption Log and open items 19
10.1 Assumption Log (first entries) 19
10.2 Open items 20
11. Questions 21
11.1 Client questions 21
11.2 Questions for the lecturer 21
Appendix A. Generative AI Usage Log 22

1. System vision

1.1 The problem

SaigonBite runs cloud kitchens ("hubs") in Ho Chi Minh City. A hub has no dining room. It cooks only for delivery, for several virtual brands. SaigonBite has several hubs in different districts and plans more.

The whole process is manual today. Orders arrive by phone, by Zalo and through third-party delivery apps. Hub staff write each order on a paper ticket and pin it to a board, so nobody outside the kitchen can see how busy the hub is. At lunch and dinner one hub is overloaded and customers wait an hour, while a nearby hub is almost idle. When an ingredient runs out the apps are not updated, and staff phone customers to cancel or replace items. Shippers are called one by one or through a group chat. Most customers pay cash on delivery, and shippers hand the money to the hub at the end of the day, sometimes with mistakes. Foreign tourists struggle to order at all: they have no Vietnamese phone number, the menu is only in Vietnamese and their foreign cards are not accepted. Discount codes are announced in chat and applied by hand. The manager handles refunds for wrong or late orders one at a time. There are no reliable reports on what sells, which hub is busy or how long deliveries take.

1.2 Vision

SaigonBite will build one online platform on which customers order and pay, hub staff and shippers work each order through a clear sequence, the platform spreads demand across hubs, and managers read reports from the same records. This proposal covers the first release, which the team will analyse, design, build and test as one integrated application with four main use cases.

1.3 Capabilities and expected benefits

The benefits below are expected outcomes, not measured results. The system records timestamps from Iteration 3 so that they can be measured.

Pain point (from the case)	Capability	Expected benefit	Use case
Orders on paper tickets, phone and chat; nobody outside the kitchen sees the queue	One order record with a fixed list of states and a named person responsible for each state	Everyone sees the same order status; no re-typing between channels	UC-01, Kitchen board, UC-02
One hub overloaded while a nearby hub is idle	Active orders counted per hub; a hub at capacity stops taking orders and the customer is offered a quieter hub; a quiet-hub incentive (a configurable delivery-fee discount)	Fewer orders queued behind a full kitchen; demand moves to hubs with spare capacity	UC-01, UC-03
Menu items run out and customers are phoned afterwards	Only items available at the chosen hub can be ordered; if an item runs out later the kitchen rejects the order or replaces the item with the customer's consent	Fewer surprise cancellations	UC-01, Kitchen board
Prices, fees and vouchers applied by hand	Full quote before confirmation: items, delivery fee based on distance, at most one voucher	The customer knows the price before paying	UC-01
Tourists cannot register, read the menu or pay with foreign cards	Local and tourist customer types with different online payment options; both can pay cash	Tourists get a payment route that exists	UC-01
Shippers reached one by one; cash handed over at the end of the day with mistakes	The shipper sees ready orders, records pickup, delivery, cash collected or failure per order; office staff reconcile the handed-over cash against those records	Cash is tied to a specific order; the end-of-shift handover has a record to check against	UC-02
No reliable reports	Sales, hub workload and delivery performance computed from the shared records	Managers compare hubs using the same definitions	UC-04

Complaints, refund decisions and customer ratings stay in the business process (Section 3) but are not built in this release (Section 5). Online payments that need a refund are stored as REFUND_PENDING. Menu translation for tourists is also outside this release.

1.4 Constraints fixed by the case

- Hubs cook only for delivery, have limited kitchen capacity and work with several brands.
- A customer is either a local customer or a foreign tourist. Registration details and online payment options differ. Both can pay cash on delivery.
- An order passes through a sequence of states from placement until it is delivered, cancelled or fails.
- Deliveries are limited to a maximum distance from the hub, and the delivery fee depends on distance.
- A customer can use at most one voucher per order.
- A busy-kitchen rule and a quiet-kitchen incentive exist. The figures will come from the client.
- Customer addresses and phone numbers are private.

2. Approach and technology

2.1 Iterative Unified Process

The team follows iterative Unified Process practices in six time-boxed iterations. Each iteration analyses, designs, builds and tests a working increment, and ends with a joint review by both students. Inside each iteration the work follows the six core processes of the textbook (chapter 1, RMO example): identify the problem and obtain approval, plan and monitor the project, discover and understand the details, design the system components, build, test and integrate, and complete system tests and deploy.

Why it suits this project:

- The case is incomplete on purpose. Capacity limits, fees, payment rules and report definitions come from the client Q&A. Short iterations let the team start on logged assumptions and change course when the answers arrive.
- The four use cases share one Order record. Building a thin working slice in Iteration 1 and growing it exposes integration problems early, when they are cheap.
- The riskiest rules are tackled first: the capacity check in Iteration 2 and the full payment and delivery paths in Iteration 3.
- The deadlines are fixed (Part 1 on 26/10, Part 2 on 16/11, Part 3 on 30/11) and the team has two people. Fixed-length iterations with an exit check make progress visible. Scrum ceremonies would add overhead for two people without adding information.

2.2 Technology stack (decided)

Part	Choice	Why
Language	Python 3.12 or later (Iteration 1 tested on 3.13)	Both students are strongest in Python.
Web framework	Flask with Jinja2 templates	Small, routes stay thin, templates map directly to screens.
Data access	sqlite3 data-access objects (DAO) with hand-written SQL, no ORM	The report needs database mapping and SQL code to be visible. An ORM would hide both.
Database	SQLite	No server to install; schema and seed data load with one script.
Testing	pytest	Test names can carry test-case IDs; output serves as the recorded result.
Login	Flask-Login with seeded users	Role checks are enforced on the server, not only by hiding buttons.

The system is one application with one database.

2.3 Three-layer design and the four roles

Layer	Role	Python package	Examples
Presentation	View	`presentation/`	One Flask blueprint per use case; Jinja2 templates
Business	Controller	`business/controllers/`	`PlaceOrderController`, `HubController`, `DeliveryController`, `ReportController`; one method per system operation in the SSD
Business	Domain	`business/domain/`	`Order`, `OrderLine`, `Hub`, `Customer` with `LocalCustomer` and `TouristCustomer`, `Payment`, `Delivery`
Data access	DAO	`data_access/`	`OrderDAO`, `HubDAO` and others, with SQL written by hand

Deployment is `python -m venv .venv`, `pip install -r requirements.txt`, `python init_db.py`, `flask run`.

3. Overall business process

A customer browses hubs and menus, picks a hub and items, and receives a quote with the delivery fee and voucher discount. If the hub is busy, the platform suggests a quieter hub and the customer chooses again. The customer pays cash on delivery (COD) or through the online stub; if online payment fails, the customer retries or switches to COD. The platform then creates the order as PLACED. While the order is PLACED or CONFIRMED the customer may cancel; cancellation is a separate small activity (Figure 1e, Cancel order), analysed but not built (A-08). The kitchen confirms or rejects the order. If an item runs out after ordering, the kitchen offers a replacement and the customer decides; if the customer declines, the order is rejected. A rejected order voids a COD payment and moves an online payment to REFUND_PENDING. The kitchen prepares the order and marks it READY, and a shipper picks it up and delivers it. On success the shipper records the COD collection and the order becomes DELIVERED. On failure the shipper reports it and returns the food; the order becomes DELIVERY_FAILED, a COD payment is voided and that path ends, while an online payment moves to REFUND_PENDING and reaches the refund decision. At the end of the shift the shipper hands the cash to the hub and office staff reconcile it. After delivery the customer receives a receipt and may rate the order, or complain about a wrong or late order; office staff then decide on a refund. Managers read reports in a separate small activity from the same shared records. Grey actions are in the process but not built in this release: rating, complaint and the refund decision and execution are deferred core features (A-09), and the Cancel order activity is analysed only, like the other supporting use cases (A-08). A flow-final symbol ends one path; an activity-final symbol ends the whole activity it belongs to.

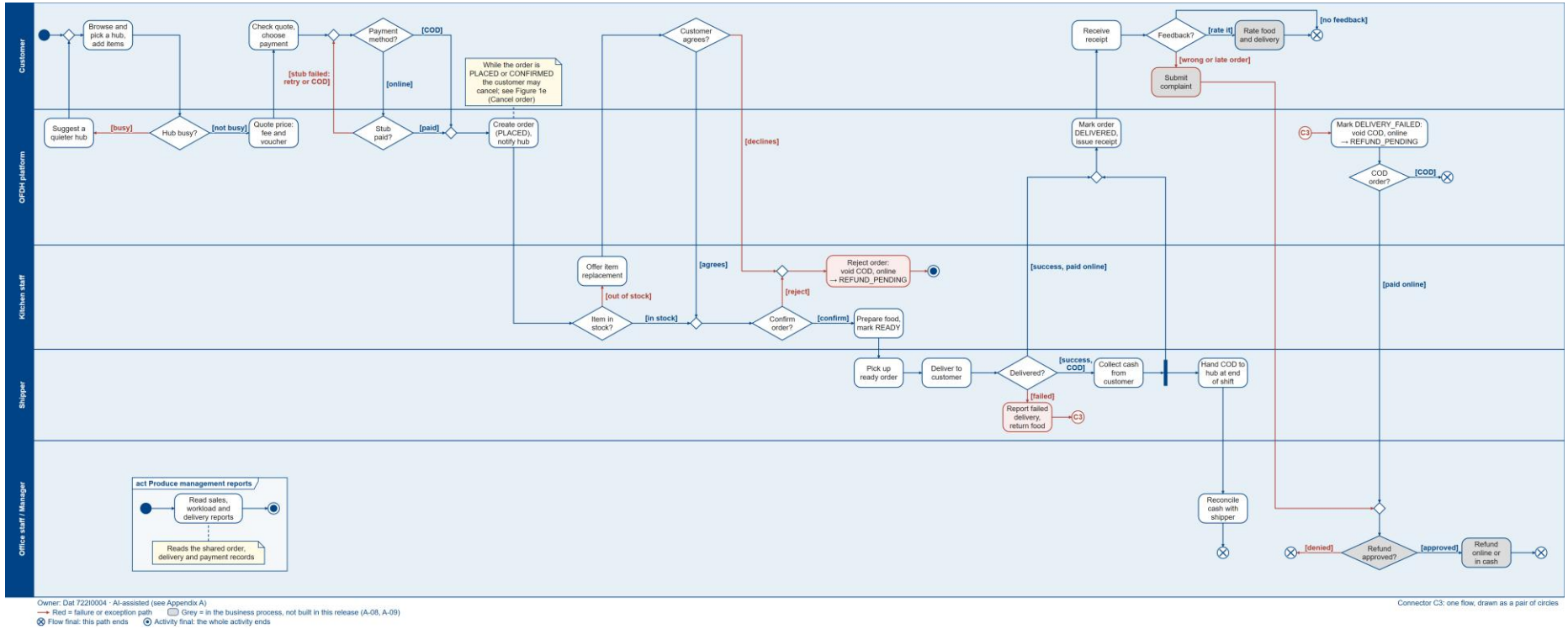


Figure 1 Overall business process (owner: Dat 722I0004; AI-assisted, see Appendix A)

3.1 Figure 1 in five parts

Figures 1a to 1e show the same process in five enlarged parts, one per page, so that the text can be read in print. Parts A to D follow the order process, and the connector codes show where the flow continues in the next part. Part E is the separate Cancel order activity, which is analysed only (A-08).

Part A: Order, quote and payment

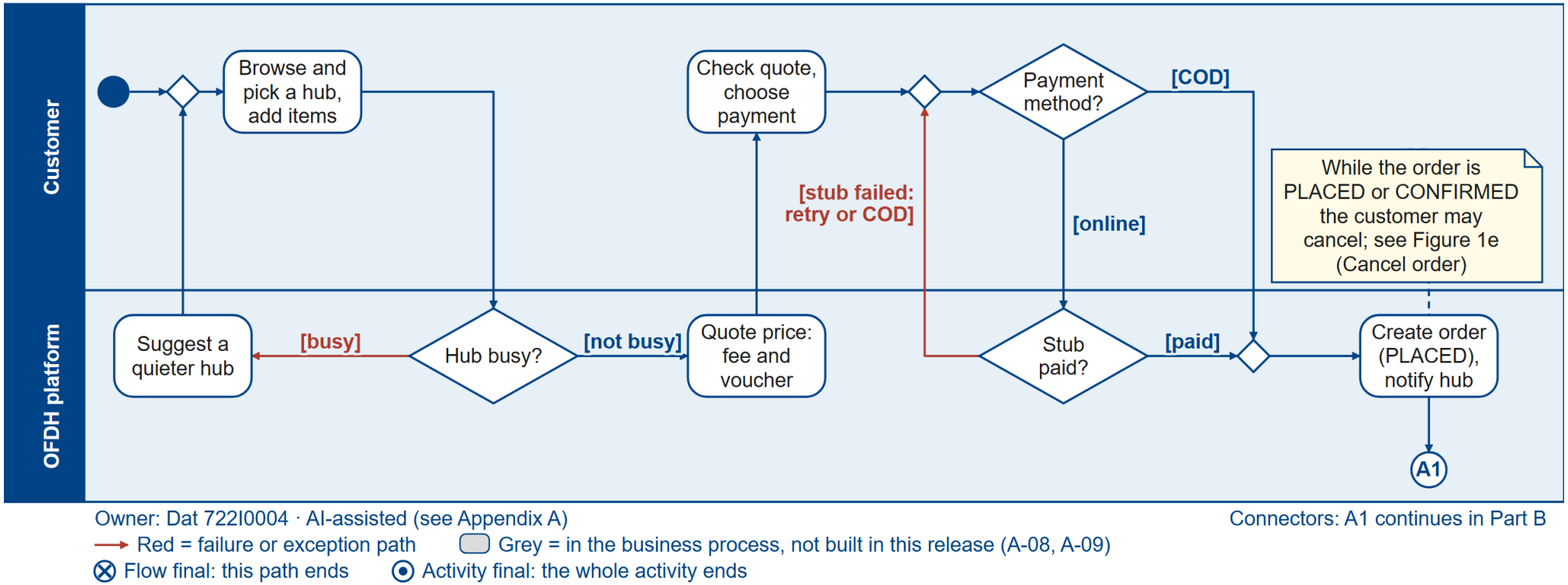


Figure 1a Overall business process, part A: Order, quote and payment (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part B: Kitchen and item replacement

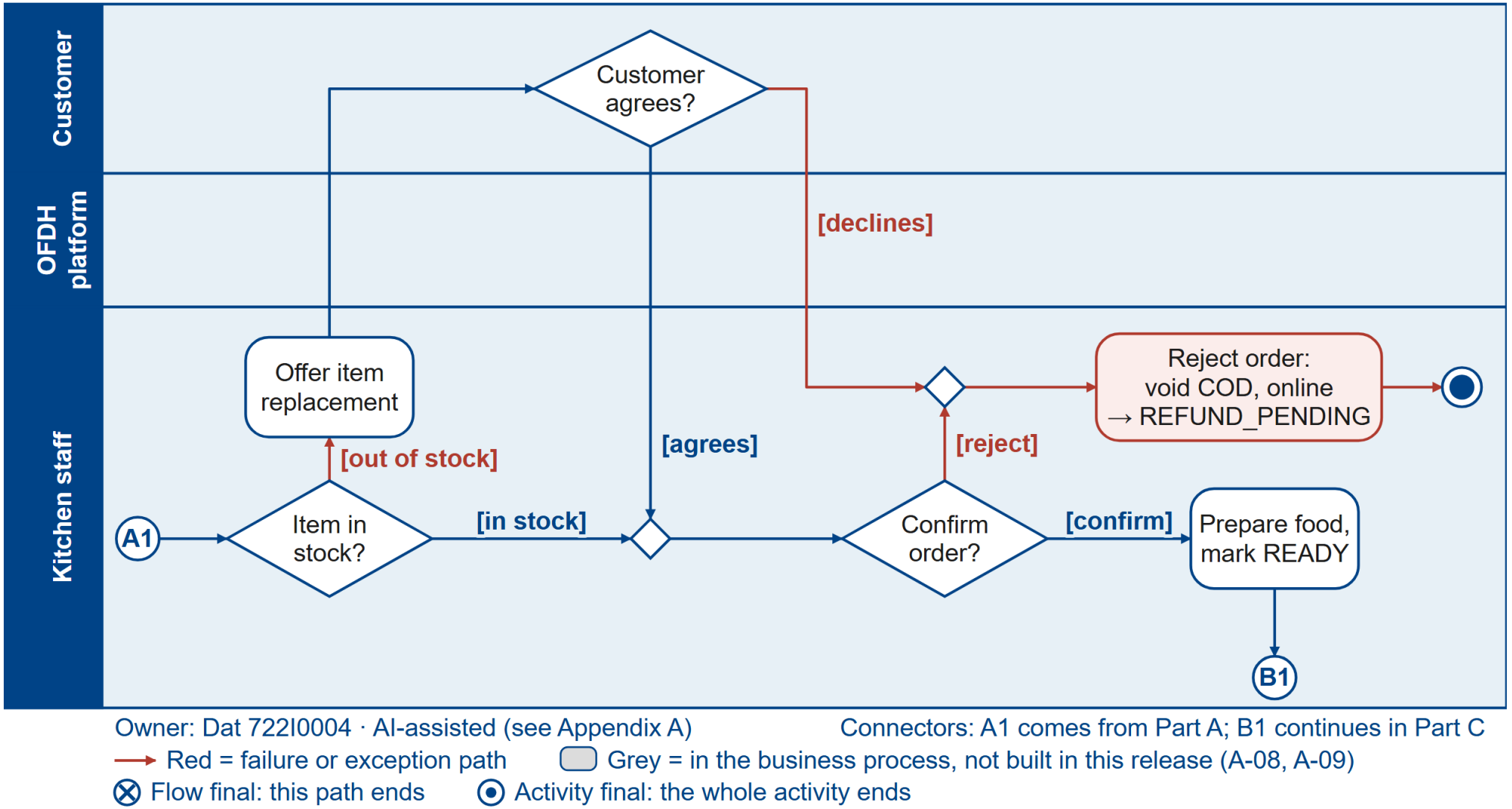
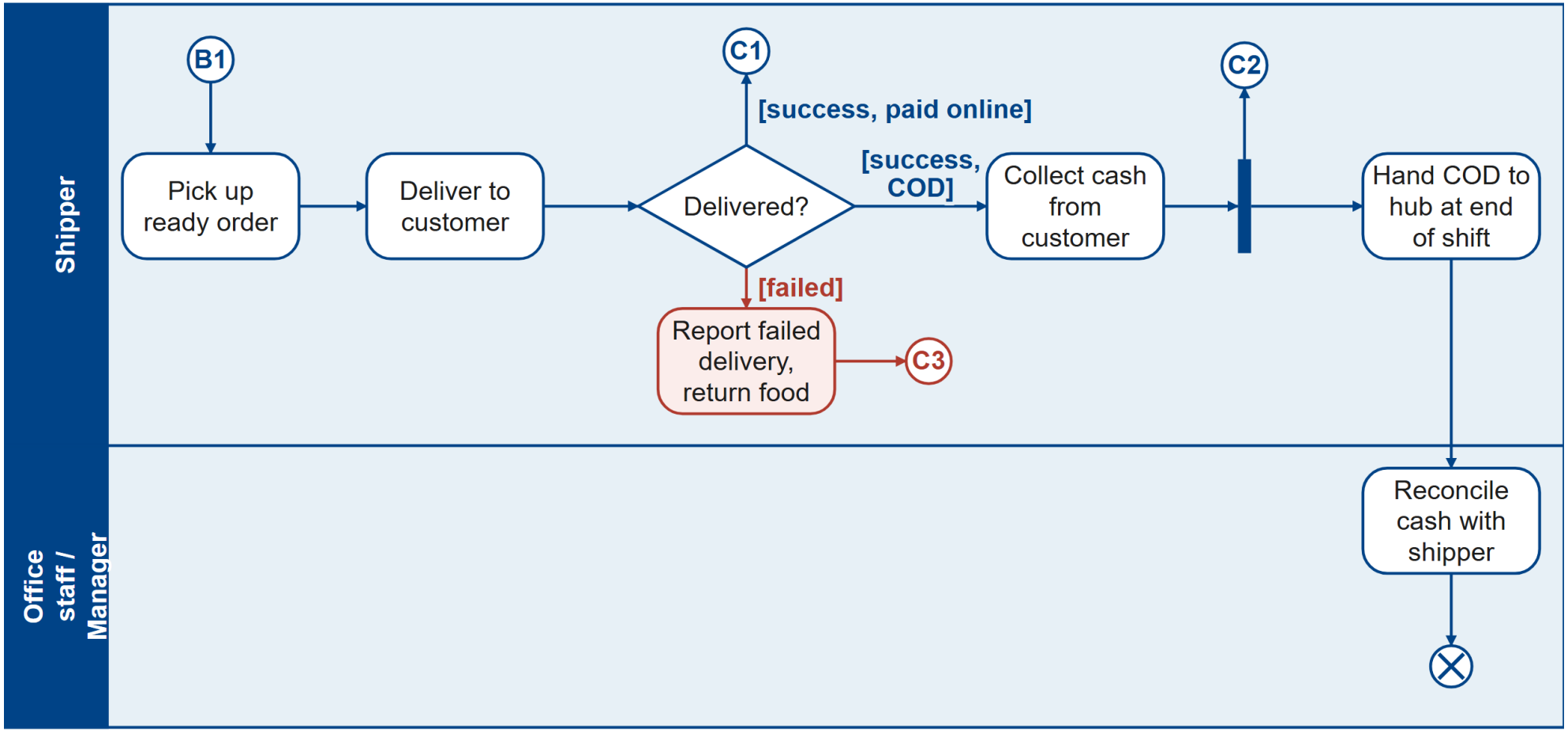


Figure 1b Overall business process, part B: Kitchen and item replacement (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part C: Delivery and cash



Owner: Dat 722I0004 · AI-assisted (see Appendix A)

→ Red = failure or exception path ▭ Grey = in the business process, not built in this release (A-08, A-09)

⊗ Flow final: this path ends ◉ Activity final: the whole activity ends

Connectors: B1 comes from Part B; C1, C2, C3 continue in Part D

Figure 1c Overall business process, part C: Delivery and cash (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part D: Receipt, failed delivery, refund and reports

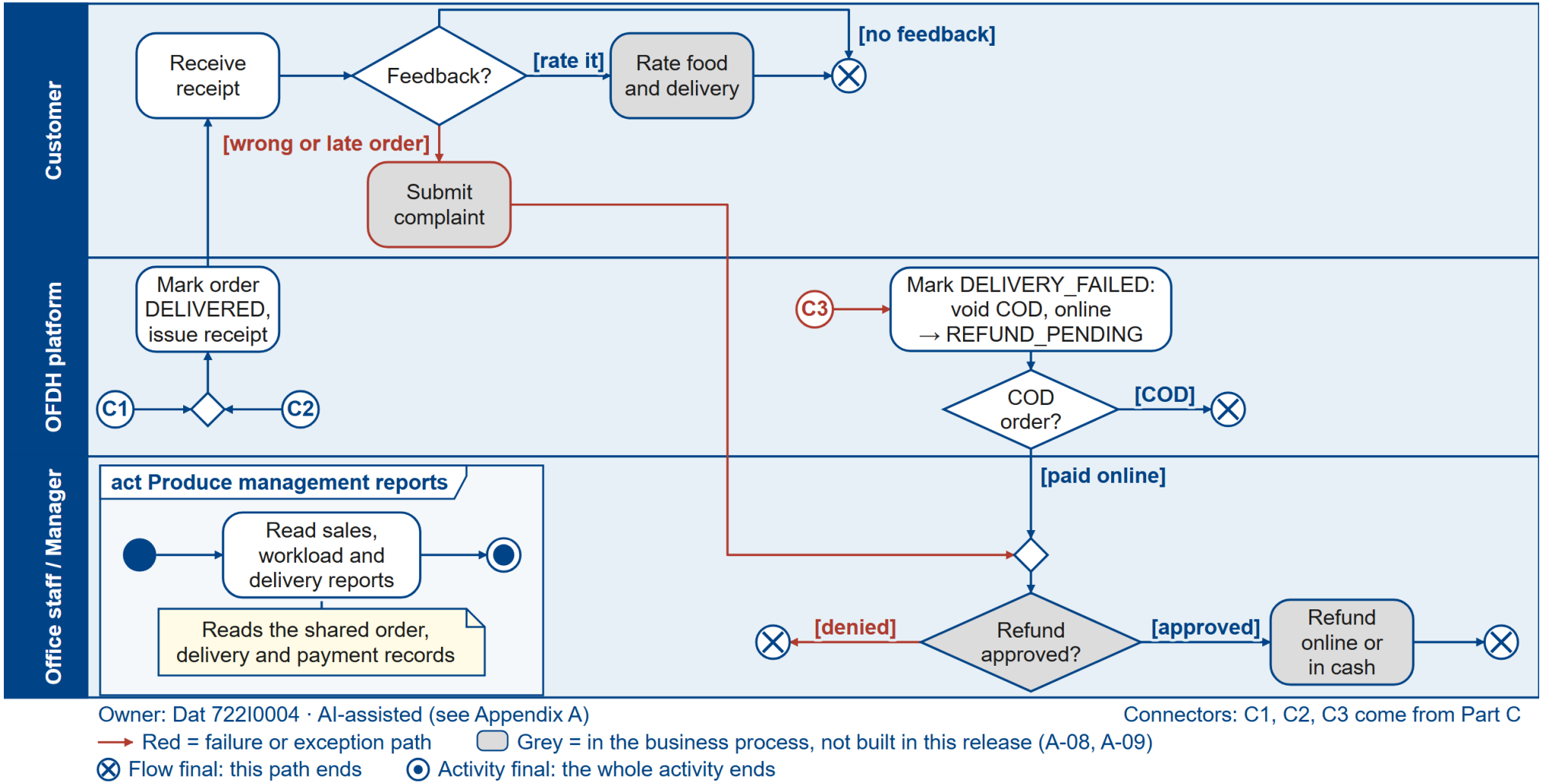
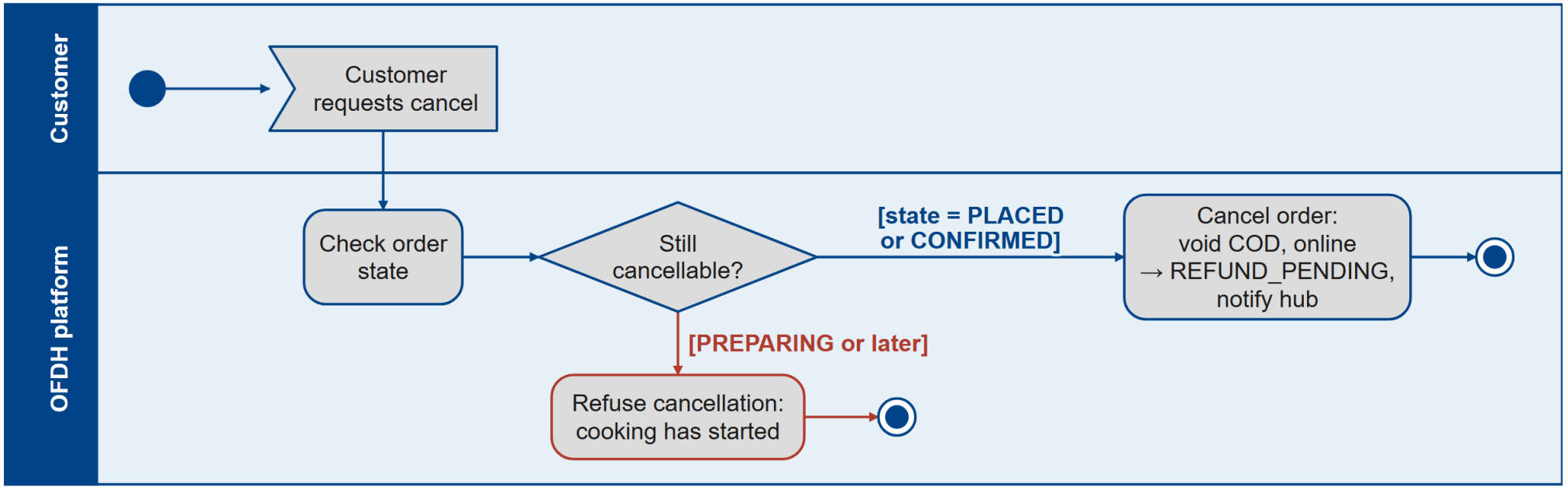


Figure 1d Overall business process, part D: Receipt, failed delivery, refund and reports (owner: Dat 722I0004; AI-assisted, see Appendix A)

Part E: Cancel order (separate activity, analysed only, A-08)



Owner: Dat 722I0004 · AI-assisted (see Appendix A)

→ Red = failure or exception path Grey = in the business process, not built in this release (A-08, A-09)

⊗ Flow final: this path ends ⊙ Activity final: the whole activity ends

Figure 1e Overall business process, part E: Cancel order, a separate activity analysed only (A-08) (owner: Dat 722I0004; AI-assisted, see Appendix A)

4. Stakeholders, actors and use case ownership

4.1 Stakeholders and actors

Stakeholder / actor	Goal	What the system must give them
Customer (local or tourist)	Order food and know the price, pay, receive the order	Quote before confirmation; payment options matching customer type; busy-hub message with a quieter hub
Kitchen staff	Cook only what the hub can handle	Incoming orders to confirm or reject; preparing and ready states
Shipper	Deliver, and account correctly for cash	Ready orders assigned to them; delivered, cash collected, or failed; a record to hand the cash over against
Admin	Keep hubs and their capacity correct	Create, view, update and archive hubs
Manager	Decide which hubs and brands need attention	Sales, hub workload and delivery performance reports
Office staff	Settle complaints fairly and check the cash	Per-order cash records to reconcile against the shipper's handover; complaint and refund decision (in the process, not built)
Payment provider	Return an online payment result	Team-built stub standing in for the provider
SaigonBite management (client)	A platform that solves the stated problems and can grow	Answers to the client questions in Section 11

4.2 Use case ownership

Each student owns exactly two main use cases. The four use cases cover groups (a) ordering, (b) fulfilment, (c) full CRUD and (d) report.

ID	Use case	Candidate #	Actor	Group	Owner	State machine
UC-01	Place an order (price it, apply a voucher, choose payment)	#3	Customer	(a) ordering	Dat 722I0004	Order
UC-02	Deliver an order (pick up, deliver, confirm cash or report failure)	#10	Shipper	(b) fulfilment	Quan 523K0049	Delivery
UC-03	Manage hubs and kitchen capacity	#13	Admin	(c) full CRUD	Dat 722I0004	Hub
UC-04	Produce management reports (sales, hub workload, delivery performance)	#16	Manager	(d) report	Quan 523K0049	none (read-only)

The owner is the author of every model of the use case, with the student ID on each diagram, and writes most of its code. The other student reviews and may help with code, and commits are recorded under the person who did the work. The Kitchen board and the login with seeded users are minimum supporting builds, owned by Quan. Dat is the integration lead: he merges the branches, owns the integration demo scenario (Section 5.2) and runs it at the end of I3 and I4. Quan keeps the shared schema and seed data so that every use case reads the same records.

Part 3 plan: Dat presents UC-01 and Quan presents UC-02.

5. Scope

Level	Items	What the team does
Built (main use cases)	UC-01, UC-02, UC-03, UC-04	Analyse, design, build and test in one application
Minimum support	Kitchen board (confirm, reject, preparing, ready); role-based login with seeded users	Build the minimum so one order can travel from placement to delivery
Analysed, not built	Register customer and addresses; manage brands and menu items; manage vouchers; manage menu availability; manage user accounts; track order; cancel or edit order (Cancel order activity, Figure 1e); archive old orders	Use case list, description and CRUD mapping; data comes from seed files. Depth pending lecturer confirmation (A-08, L1)
Deferred core features	Handle complaint and refund (including executing a refund); rate an order	Kept in the business process (grey actions in Figure 1), use case list and backlog; not built in this release (A-09, L3)
Future (from the case)	Loyalty points, subscription meal plans, group orders, scheduled orders	Mentioned in the vision and conclusions only

Order states are fixed: PLACED, CONFIRMED, REJECTED, PREPARING, READY, OUT_FOR_DELIVERY, DELIVERED, DELIVERY_FAILED, CANCELLED. An order being DELIVERED is not the same as the payment being PAID. The Delivery has its own states in Quan's state machine; when the delivery is PICKED_UP, the order moves to OUT_FOR_DELIVERY.

Payment states are listed here once and used the same way in the process, A-06, Q3 and the slides:

Payment state	Meaning
PENDING	A COD payment waits for the cash to be collected
PAID	Online payment succeeded, or the shipper confirmed that the cash was collected
FAILED	The stub returned failure for one online attempt; no order is created, so this is shown to the customer and logged only
VOIDED	A COD payment that will never be collected (kitchen rejection, cancellation or failed delivery); no money moved
REFUND_PENDING	An online payment that must be given back after a rejection, cancellation or failed delivery; stored, but the refund itself is not executed in this release (A-09)

REFUNDED (the refund was executed) is analysed, not built.

5.1 CRUD note

After CRUD verification every persistent class must be created, read, updated and deleted (or archived) by some use case. The team's mapping:

Class	Covered by	Built?
Hub	UC-03: create, read, update, archive (guard: no active orders)	Yes: create, read, update, archive
Order, OrderLine	Create: UC-01. Read and update: Kitchen board, UC-02. Archive: the supporting use case "archive old orders"	Create, read, update yes; archive analysed, not built
Payment, Delivery	Create and update: UC-01 and UC-02. Read: UC-04. Archive follows the Order	Create, read, update yes; archive analysed, not built
Customer and Address, Brand and MenuItem and HubMenuItem, Voucher, UserAccount and Shipper	Supporting use cases analysed in the list above	No; seed data

Design for the analysed archive use case: when it is built, an archived order carries an `archived_at` timestamp. It disappears from operational screens, reports still read it, and nothing is physically deleted. Payment and Delivery are filtered through the Order's flag. The lecturer is asked whether analysis plus seed data is acceptable for the unbuilt classes (L1, Section 11).

Fallback if the lecturer rejects analysis plus seed data (L1): the team builds one generic list, create, edit and archive admin pattern for the classes the lecturer names, cheapest first (Voucher, MenuItem, Brand). The

estimate is about 12 person-hours per class, scheduled in I3 and I4. It covers up to two classes (about 24 person-hours), paid for by the I3 to I5 buffers (24 person-hours, Section 6.2) and by moving Should and Could items (US-17, US-20) out of the plan. A third class would exceed the buffers, so the team would ask the lecturer which class stays analysis-only. The decision deadline is Sat 31/10, the end of I2.

5.2 Integration demo scenario

1. The Admin lowers the capacity of a seeded hub that already has a menu (the District 1 hub) (UC-03).
2. A customer places orders until the hub is busy, is blocked and is offered a quieter hub (UC-01).
3. Kitchen staff confirm the orders and move them to preparing and ready.
4. A shipper delivers one COD order and reports one failed delivery (UC-02).
5. The Manager reads the reports; the figures match a table the team calculated by hand beforehand (UC-04).

Create Hub and Archive Hub are tested separately. A new hub has no menu, so it cannot take orders in the demo.

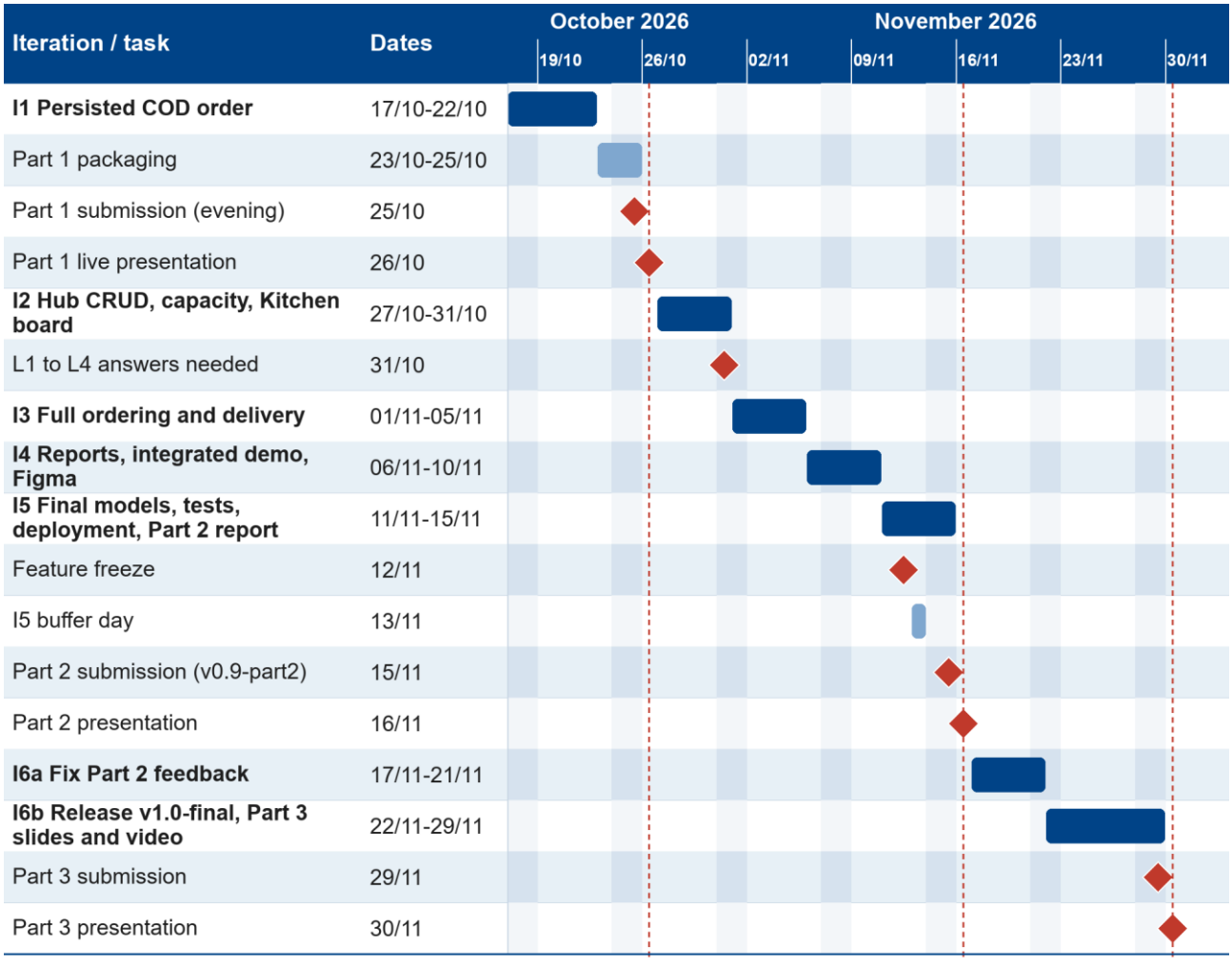
6. Six-iteration roadmap

Assumed effort is about 4 focused hours per student per day, so Iteration 1 is 2 x 6 x 4 = 48 person-hours. The team re-checks this figure after Iteration 1 and re-plans the hours of I2 to I5 if it was wrong (Section 6.2). The lecturer questions L1 to L4 are asked at the Part 1 presentation on 26/10 and the team needs the answers by Sat 31/10, the end of I2. Nothing planned for Part 2 relies on I6: Iteration 5 holds its own one-day buffer.

Iteration and dates	Goal and increment	Use cases and stories	Deliverables	Owner: Dat / Quan	Exit criterion
I1 Sat 17/10 - Thu 22/10 (6 days)	A persisted COD order on a runnable skeleton	UC-01 subset (one hub, COD, flat fee); US-01, US-12 (login seed)	Vision, backlog, UC-01 subset description, activity, SSD, Order state machine v1, checkout sketch; schema, seed, DAO; checkout slice; pytest results; run instructions	Dat: vision, UC-01 subset analysis and checkout slice. Quan: events and actors, domain and schema, DAO, seed, UC-02 draft, second-machine setup	Acceptance criteria AC-1 to AC-9 (Section 7.2) checked on a second machine
Part 1 packaging 23/10 - 25/10	Proposal, slides, rehearsal		Proposal PDF, 15 slides, two timed rehearsals	Both	Proposal submitted 25/10 evening; presented live 26/10
I2 Tue 27/10 - Sat 31/10	Hub CRUD, capacity rule, Kitchen board	UC-03; busy-hub check in UC-01; UC-02 analysis; US-03, US-04, US-05, US-13	UC-03 artefact set and tests; Hub state machine; UC-02 analysis and Delivery state machine; Kitchen board	Dat: UC-03 and the workload check in checkout. Quan: UC-02 analysis, Kitchen board. L1 to L4 answers needed by Sat 31/10	Boundary test passes (capacity minus 1 accepted, capacity blocked, two simultaneous orders do not both pass); an order moves PLACED to READY on the board; tag `v0.2`
I3 Sun 01/11 - Thu 05/11	Full ordering and delivery	UC-01 complete; UC-02; US-01 (full), US-02, US-06, US-07, US-08, US-14, US-15, US-16	Voucher, distance fee, quiet incentive (minimal), payment options and stub, tourist customer; UC-02 design, code, tests; timestamps	Dat: voucher, distance fee, quiet incentive (one configurable fee discount), payment options and stub, tourist. Quan: UC-02 design, code, tests, timestamps	One order ID travels from placement through delivery; one failed delivery handled; repeated submit creates one order; tag `v0.3`

Iteration and dates	Goal and increment	Use cases and stories	Deliverables	Owner: Dat / Quan	Exit criterion
I4 Fri 06/11 - Tue 10/11	Reports, integrated demo, Figma	UC-04; US-09, US-10, US-11; all four integrated	UC-04 artefacts; expected report values calculated by hand first; Figma for UC-01 and UC-03 (Dat) and UC-02 and UC-04 (Quan)	Quan: UC-04. Dat: integration lead, Figma UC-01 and UC-03, missing exceptions	Demo scenario (5.2) runs end to end; report output equals the hand-calculated table; tag `v0.4`
I5 Wed 11/11 - Sun 15/11	Final models, system tests, deployment, Part 2 report. Feature freeze Thu 12/11; buffer day Fri 13/11; report finalised Sat 14/11; submit Sun 15/11	All four; `v0.9-part2`	Final design models, database mapping and SQL, test plan with recorded results, deployment instructions, Part 2 report PDF	Both; Dat leads integration	Code frozen 12/11; tag `v0.9-part2`; fresh-machine run passes; report submitted 15/11
I6 Tue 17/11 - Sun 29/11	Fix Part 2 feedback, release `v1.0-final`, Part 3	All four	Corrections to Part 2 feedback, `v1.0-final` release, Part 3 slides and video, contribution statement, AI log	Both	Release tagged; Part 3 package submitted 29/11

6.1 Gantt chart



Dark bar = iteration. Light bar = task. Diamond = milestone. Dashed lines = Part 1, Part 2 and Part 3 presentation days. Grey columns = weekends.

Figure 2 Roadmap from 17/10 to 30/11/2026 with the Part 1 (26/10), Part 2 (16/11) and Part 3 (30/11) presentation days

I6a and I6b together form I6 (17/11 to 29/11).

6.2 Planned effort and buffer

Baseline: 4 focused hours per student per day. Build hours are code, automated tests, seed and deployment scripts; the rest of each person's time goes to analysis, design, documents and cross-review, where Quan carries more (domain model, schema, UC-02 and UC-04 models) and Dat carries the vision, backlog and process diagram. The buffer is reserved time that is not planned to any story.

Iteration	Days	Capacity (person-hours)	Dat planned	Quan planned	Buffer	Build hours Dat / Quan
I1	6	48	24	24	0	12 / 8
Part 1 packaging	3	24	10	10	4	0 / 0
I2	5	40	18	18	4	10 / 6
I3	5	40	16	16	8	12 / 8
I4	5	40	16	16	8	8 / 5
I5	5	40	16	16	8 (Fri 13/11)	5 / 3
I6	13	104	40	40	24	12 / 7
Total	42	336	140	140	56	59 / 37

Dat carries about 61 % of the build hours (59 of 96) and Quan about 39 % (37 of 96), which matches ownership: Dat builds UC-01, UC-03 and the integration, Quan builds UC-02, UC-04, the Kitchen board, schema and seed. The I1 row is fully planned because its six days are fixed by the requirements; the figures for I2 to I6 are re-checked after I1 against the hours actually logged (Section 7.3). The I3 to I5 buffers (24 person-hours) also pay for the CRUD fallback if L1 is rejected (Section 5.1). Should and Could items (US-17, US-20) are worked only from buffer time.

7. Iteration 1 in detail

7.1 Day-by-day plan

Each day is about 4 focused hours per student. The days follow the six core processes of textbook chapter 1. Repository, Flask skeleton and `init_db.py` are set up on Day 1 as part of CP2.

Day and date	Textbook core process	Dat (UC-01)	Quan (shared foundation and UC-02)	End-of-day evidence
D1 Sat 17/10	CP1 identify the problem and obtain approval; CP2 plan and monitor the project	Write the vision from the case; list the problem-to-capability table; write the UC-01 user stories; MoSCoW backlog v1; risk list v1	Create the repository and Flask skeleton with the four packages; `init_db.py` creating an empty database; events and actors v1; use case diagram v1	Backlog v1; skeleton runs with `flask run`; approval path for the scope noted (lecturer feedback arrives on 26/10)
D2 Sun 18/10	CP3 discover and understand the details	UC-01 subset description (COD, one hub): pre- and post-conditions with IDs, quote rules; five client questions written; business process diagram (Figure 1) v1	Domain model v1; CRUD inventory v1	Class and order state names fixed; UC-01 description complete
D3 Mon 19/10	CP3 and CP4 design the system components	UC-01 activity diagram, SSD, Order state machine v1, checkout sketch	`schema.sql` and `seed.sql` (hub with menu, customer, users per role); UC-02 description draft; check Dat's SSD against the schema	SSD operations, schema columns and state names agree

Day and date	Textbook core process	Dat (UC-01)	Quan (shared foundation and UC-02)	End-of-day evidence
D4 Tue 20/10	CP4 and CP5 build, test and integrate	Design UC-01 in four roles; `PlaceOrderController` and `Order` domain code	DAO base; Hub and menu reads; `OrderDAO` insert; review the UC-01 design	A COD order is saved by a script or test
D5 Wed 21/10	CP5 build, test and integrate	Checkout page; pytest cases for UC-01 (success, invalid input, repeated submit)	Integration through the DAOs; role checks on the server; README run steps	pytest run recorded with date and commit
D6 Thu 22/10	CP6 complete system tests and deploy	Fix defects; run UC-01 acceptance scenarios; update risks and assumptions	Run the README on a second machine and execute the same scenarios; revise the roadmap using I1 hours	Second-machine result; I1 review held; plan vs actual (7.3) filled in

7.2 Iteration 1 acceptance criteria (planned)

ID	Criterion
AC-1	From a clean checkout on a second machine, the four setup commands run without manual edits and the home page opens.
AC-2	`init_db.py` loads at least two hubs with brands and menu items, one local customer with an address, and one user for each of the roles customer, kitchen staff, shipper, admin and manager.
AC-3	A logged-in customer selects a hub, items, quantities and an address, and sees item subtotal, delivery fee and total before confirming. The fee is a flat seeded value until the client answers the fee question.
AC-4	Confirming creates one Order in state PLACED with its lines and a COD Payment in a single transaction. The confirmation shows the order ID and total.
AC-5	Reading that order back through a new database connection returns the same lines and the same total.
AC-6	An empty cart, quantity of zero or less, an item not available at the hub, and a missing address each produce a clear message and no new order.
AC-7	Pressing confirm twice creates exactly one order.
AC-8	A user who is not logged in is redirected to login, and a user without the customer role cannot place an order, checked on the server.
AC-9	pytest covers AC-4 to AC-8 with results recorded. The UC-01 description, activity diagram, SSD, Order state machine and schema use the same names.

7.3 Plan vs actual

Actual values come from the repository history, the pytest output and the hours log of 17/10 to 22/10. The source repository is https://github.com/Dat071104/SaigonBite (tag `i1-evidence`). The Iteration 1 code skeleton, schema and seed, UC-01 subset and pytest suite were generated with AI assistance (Claude, 19/10);

the team's review of that code is recorded in Appendix A. The table therefore states for each day what was finished and by whom (team or AI-assisted).

Day	Planned evidence	Actual	Variance and action
D1 Sat 17/10	Backlog v1; skeleton runs	Planning only: plan v1/v2 (15/10, Claude-assisted); vision, backlog, risks in plan v3 (19/10). No repository or skeleton.	Repository and skeleton missing on D1; build started on D3 (19/10), recovered with AI-assisted scaffolding; team review still to do.
D2 Sun 18/10	Names fixed; UC-01 description	Client questions and CRUD table in plan v3 (19/10); UC-01 description, domain model, CRUD inventory drafted by AI on 19/10.	No separate D2 files existed; drafts written on D3 with AI assistance, names fixed in code and schema; team review still to do.
D3 Mon 19/10	SSD, schema and states agree	AI-assisted session 19/10: skeleton, schema and seed, UC-01 subset, 81 pytest tests, five analysis docs. No activity diagram.	Activity diagram and checkout sketch (Dat), UC-02 draft (Quan) still missing; AI-built code and docs not yet human-reviewed.
D4 Tue 20/10	COD order saved	Done early on 19/10 (commits 622549a, 2b11e42, 6595553): domain, DAOs, controller; COD order saved by tests and real-server run.	Built ahead of plan on D3, AI-assisted; Dat and Quan have not yet reviewed the code; line-by-line review to do.
D5 Wed 21/10	pytest run recorded	Done early on 19/10 (6d4c61a, e35bc29, 2e4eecf, 049673d): views, README, 81 pytest tests; run recorded: 81 passed.	Ahead of plan but AI-generated; role checks and README steps not yet verified by Quan or by a second person.
D6 Thu 22/10	Second-machine run; review	⟦ACTUAL d6_done: what was finished on D6, and by whom (team or AI-assisted)⟧	⟦ACTUAL d6_var: variance and action for D6⟧

Measure	Plan	Actual
Person-hours (human, Dat and Quan)	48	⟦ACTUAL hours_human: hours logged by Dat and Quan, separate from AI-assisted generation⟧
Acceptance criteria met	9 of 9	7 of 9 met; AC-1 (same-machine clone only) and AC-9 (no activity diagram, no review) partial, 19/10
pytest result	AC-4 to AC-8 covered	81 passed, 0 failed - 19/10/2026, commit 049673d (tag i1-evidence)
Second-machine run (Quan's laptop)	Passes	⟦ACTUAL second_machine_result: result and problems found on Quan's laptop⟧
Change to the roadmap	None expected	⟦ACTUAL i1_roadmap_change: change made after I1, if any⟧
Lesson that changed the plan	None expected	⟦ACTUAL lesson_learned: the lesson from Iteration 1⟧

8. Prioritised backlog

MoSCoW priority. A story is Must when the main use case cannot work without it, or when the case states it as a fixed rule (voucher, payment options by customer type, busy and quiet kitchen). Should and Could items are worked only from buffer time (Section 6.2).

ID	User story	Priority	Iteration	Owner
US-01	As a customer, I want to see the full quote and confirm a COD order so that I know what I will pay.	Must	I1, completed in I3	Dat
US-02	As a customer, I want delivery refused beyond the maximum distance and the fee based on distance.	Must	I3	Dat
US-03	As a customer, I want a busy hub to refuse my order and suggest a quieter hub, so that I do not wait an hour.	Must	I2	Dat
US-04	As an admin, I want to create, view and update a hub and its capacity.	Must	I2	Dat
US-05	As an admin, I want to archive a hub that has no active orders so that history is kept.	Must	I2	Dat
US-06	As a shipper, I want to pick up a ready order assigned to me.	Must	I3	Quan
US-07	As a shipper, I want to confirm delivery and cash collected.	Must	I3	Quan
US-08	As a shipper, I want to report a failed delivery with a reason.	Must	I3	Quan
US-09	As a manager, I want a sales report with a date filter.	Must	I4	Quan

ID	User story	Priority	Iteration	Owner
US-10	As a manager, I want a hub workload report.	Must	I4	Quan
US-11	As a manager, I want a delivery performance report.	Must	I4	Quan
US-12	As a user, I want access limited by role and by the records assigned to me.	Must	I1 to I5	Quan
US-13	As kitchen staff, I want to confirm, reject, prepare and mark orders ready.	Must	I2	Quan
US-14	As a customer, I want to apply one voucher and see the discount.	Must	I3	Dat
US-15	As a customer of a quiet hub, I want a fee discount so that I choose it. Minimal mechanism: one configurable incentive, a delivery-fee discount for a hub under the quiet threshold; threshold and amount sit in configuration until Q1 is answered.	Must	I3	Dat
US-20	As the business, I want advanced quiet-hub incentive variants (time windows, stacking with a voucher).	Should	I4 or I5 buffer, only if time allows	Dat
US-16	As a local customer or tourist, I want online payment options for my customer type through the stub, and to retry or choose COD if it fails.	Must	I3	Dat
US-17	As a customer, I want to follow my order's status.	Could	I4, only if ahead of plan (buffer time)	Dat
US-18	Handle complaint and refund; rate an order; register customer and addresses; manage brands, menu, vouchers, menu availability and user accounts; cancel or edit order; archive old orders.	Won't (this release; stays in process and use case list)	n/a	n/a
US-19	Loyalty points, subscription meal plans, group orders, scheduled orders.	Won't (future)	n/a	n/a

9. Top 5 risks

ID	Risk	Trigger	Likelihood / impact	Owner	Mitigation
R-01	CRUD or scope coverage is rejected, so unbuilt classes must be built	Lecturer rejects the CRUD mapping (5.1)	Medium / High	Dat	Ask L1 on 26/10 and need the answer by 31/10; fallback is one generic admin pattern at about 12 person-hours per class, built in I3 and I4 from the I3 to I5 buffers (up to two classes); the I5 buffer day is Fri 13/11
R-02	Business rules are unconfirmed or change after coding	Client answers differ from the proposed defaults	High / Medium	Dat	Keep every proposed rule in the Assumption Log; keep thresholds and fees in configuration; trace each answer to the tests it changes

ID	Risk	Trigger	Likelihood / impact	Owner	Mitigation
R-03	Peak-hour capacity race: two orders are accepted past capacity, or order states drift	Boundary test or concurrent-order test fails	Medium / High	Dat	Count active orders and insert the order in one transaction (SQLite `BEGIN IMMEDIATE`); boundary test at capacity minus 1, capacity, and two simultaneous orders
R-04	Integration drift between use cases: different IDs, state names or separate data	A use case needs its own copy of an order or a state not on the fixed list	Medium / High	Dat (integration lead)	One shared schema and seed from I1, kept by Quan; one Order flows through every use case; state names frozen after I1; Dat runs the demo scenario at the end of I3 and I4; cross-review every iteration
R-05	A member is unavailable or late	A day's planned evidence is missing at the daily check	Medium / High	Quan	Cross-review so each knows the other's use case; both can run the repository; per-iteration buffers and the I5 buffer day (Fri 13/11), with the feature freeze on 12/11; drop Should and Could items first; re-plan hours after I1

10. Assumption Log and open items

10.1 Assumption Log (first entries)

ID	Assumption	Reason	Affected work	Owner	Status / confirmed by + date
A-01	One order is cooked by one hub.	Keeps responsibility for the order clear.	UC-01, domain model	Dat	Proposed; team default, open to client correction
A-02	Hub workload = orders in PLACED, CONFIRMED and PREPARING. A hub is busy when this count is at or above its `max_active_orders`. The check runs once, when the order is created.	The case gives no figure; one definition is used by checkout and by the workload report.	UC-01, UC-03, UC-04	Dat	Proposed; awaiting client (Q1)
A-03	The quiet-hub incentive is one configurable delivery-fee discount for a hub under the quiet threshold; threshold and amount stay open and sit in configuration.	The case says the figures will come from the client.	UC-01	Dat	Proposed; awaiting client (Q1)
A-04	Distance is the straight-line (Haversine) distance from seeded coordinates. The maximum radius and the fee formula are open.	No map service is used; the formula is the client's decision.	UC-01	Dat	Proposed; awaiting client (Q2)
A-05	Payment is COD for everyone plus an online team-built stub. Local customers get e-wallet or domestic bank QR; tourists get international card.	The case says the options differ but not what they are.	UC-01	Dat	Proposed; awaiting client (Q4) and lecturer (L2)

ID	Assumption	Reason	Affected work	Owner	Status / confirmed by + date
A-06	Payment outcomes: online payment fails, no order is created; kitchen rejects, the customer cancels or delivery fails, then a COD payment is VOIDED and an online payment becomes REFUND_PENDING (states in Section 5).	A refund is not executed in this release, so the outcome must be stated.	UC-01, UC-02	Dat	Proposed; awaiting client (Q3)
A-07	Report definitions: sales are DELIVERED orders by delivered date; workload is a snapshot of active load against capacity; waiting for pickup is READY to PICKED_UP and delivery time is PICKED_UP to DELIVERED; failure rate is failed over delivered plus failed.	Expected values are calculated from these definitions before coding.	UC-04	Quan	Proposed; awaiting client (Q5)
A-08	The supporting use cases in Section 5, including cancel or edit order (the Cancel order activity, Figure 1e, grey), are analysed but not built; CRUD coverage of their classes is by analysis and seed data.	The build is limited to the four main use cases and the minimum support.	Sections 3 and 5.1, Part 2 CRUD verification	Both	Proposed; awaiting lecturer (L1)
A-09	Complaint and refund (including executing a refund) and rating are deferred core features: they stay in the process (grey in Figure 1), the use case list and the backlog, but are not built in this release, so online payments wait in REFUND_PENDING.	Keeps the scope to what can be analysed, built and tested in six iterations.	Section 3, scope	Both	Proposed; awaiting lecturer (L3)
A-10	Part 1 is presented live.	The lecturer approved a live presentation in place of the recorded video (verbal approval).	Presentation	Both	Confirmed verbally by the lecturer (before 19/10/2026)

10.2 Open items

- Source repository: https://github.com/Dat071104/SaigonBite (tag `i1-evidence`).
- Effort: about 4 focused hours per student per day is an assumption (Section 6.2), re-checked against the Iteration 1 hours (Section 7.3).
- Client Q&A: Iteration 1 is built on the assumptions above. Each client answer updates the Assumption Log and the tests it affects in the next iteration.

11. Questions

11.1 Client questions

Each question asks for one decision. Where a second point depends on it, the team states the default it will use if no answer comes.

#	Question (one decision)	Proposed default
Q1	At what number of active orders should a hub stop taking new orders?	Busy when active orders (PLACED, CONFIRMED, PREPARING) are at or above the hub's capacity. For the quiet-hub incentive (a delivery-fee discount), the trigger and amount stay configurable placeholders until you give figures.
Q2	How should the maximum delivery distance and the fee be calculated?	Straight-line kilometres from the hub; fee is a base fee plus a rate per kilometre; the team needs the maximum distance and the rates.
Q3	Up to which order state may a customer cancel?	While the order is PLACED or CONFIRMED, not yet PREPARING. After a kitchen rejection, a cancellation or a failed delivery, a COD payment is voided and an online payment becomes REFUND_PENDING.
Q4	Which online payment options should local customers and tourists each have?	Local: e-wallet or domestic bank QR. Tourist: international card. COD counts as paid when the shipper confirms collection.
Q5	Should sales reports use the delivered date or the placed date?	Delivered date. Delivery time is reported as two intervals, READY to PICKED_UP and PICKED_UP to DELIVERED.

11.2 Questions for the lecturer

The team asks L1 to L4 at the Part 1 presentation on 26/10 and needs the answers by Sat 31/10, the end of I2, so that any extra build fits before the Part 2 report.

#	Question	Related
L1	Is analysis plus seed data acceptable for the CRUD coverage of the unbuilt supporting use cases (mapping in 5.1)? If not, which classes must be built?	A-08, R-01
L2	Is a team-built payment stub acceptable for online payment?	A-05
L3	May complaint and refund and rating be deferred from the build while staying in the process and use case list?	A-09
L4	Is Python (3.12 or later; Iteration 1 tested on 3.13) acceptable as the implementation language, since the report template gives examples in other languages?	Section 2.2

Appendix A. Generative AI Usage Log

The team keeps the full prompts and outputs and will hand them over within 48 hours if asked.

Tool and date	Purpose	Prompt (summary)	Kept or changed	How verified
ChatGPT ("Sol"), 13/10/2026	Initial analysis of the case and the requirements	Asked for candidate classes, events, use cases and open questions from the case study and requirements	Used as a reading aid and a checklist of ambiguities; the team chose its own four use cases	Compared with the case study text and the requirements document
Claude, 15/10/2026	Team plan v1 and v2	Asked for scope, use case split, calendar, CRUD mapping, payment and report rules for two students	Plan v2 is the team's working plan; use case split, scope and calendar were decided by the team	Checked against the requirements, rubric and case; decisions recorded in the plan
ChatGPT, 15/10/2026	Part 1 draft package and an audit of it	Asked for a proposal, slides and speaker scripts, then for an audit against the rubric	Problem-to-capability table and Iteration 1 acceptance criteria idea reused; the rest replaced	Reviewed against the requirements and marking sheet
Claude, 19/10/2026	Strict review of the 15/10 package	Asked to find errors, meta text, undecided choices and rubric gaps	Findings used as input to the 19/10 rebuild	Each finding checked against the package and the requirements
Claude (Opus and Sonnet subagents), 19/10/2026	Rebuild of the proposal, slides, process diagram and plan v3 from the team plan, then a consistency and rubric check	Gave the team plan, case, requirements and a brief with fixed facts; asked for proposal text, slide content, the swimlane diagram and a check against the rubric	Proposal text, slides and Figure 1 generated from the team plan and brief; the team reviews, edits and must be able to explain every part before submission. Team review on ⟦ACTUAL review_date: date of the team review⟧; main edits: ⟦ACTUAL proposal_edits: main edits the team made to the proposal, slides and diagram⟧	The team checks every section against the requirements, the case and the brief, and runs two timed rehearsals on ⟦ACTUAL rehearsal_dates: dates of the two timed rehearsals⟧
Claude (Sonnet subagent), 19/10/2026	Iteration 1 code in the local repository	Asked for the code skeleton, schema and seed, the UC-01 subset and a pytest suite for the acceptance criteria AC-1 to AC-9	Generated the Iteration 1 code skeleton, schema and seed, UC-01 subset and pytest suite in the local repository; the team reviewed and changed: ⟦ACTUAL ai_code_review: what the team reviewed and changed⟧	pytest run and fresh-clone run recorded in Section 7.3; team review on ⟦ACTUAL review_date: date of the team review⟧